



Reports that recover without writing twice.

PepperBall reporting reliability - Thomas Ryan

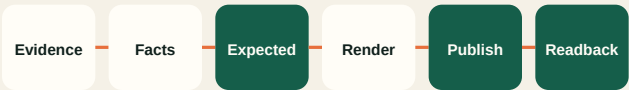
Forward Deployed Software Engineer Intern | Python, cloud, data, documents, release engineering

Writer topology Exactly one	Equivalent replay Zero writes	Lost response GET-only settlement	Regression Exact rollback
--------------------------------	----------------------------------	--------------------------------------	------------------------------

THE CHALLENGE

A scheduled manufacturing reporting workflow had to absorb late inputs, changing source documents, lost HTTP responses, and long-lived repair obligations without creating duplicate files or overwriting protected work.

RELIABILITY LOOP



REPRESENTATIVE RECOVERY

A replay once differed at the package level because capture-only metadata changed. The governed business material did not. The correction excluded those exact control fields from material identity while preserving genuine source changes. An equivalent cycle then produced no update; a later real change advanced the same item once.

WHAT I BUILT

- Durable expected-output accounting
- Governed material identity
- Deterministic two-sheet reports
- Conditional same-item publication
- GET-only accepted-write recovery
- Fair bounded repair with terminal holds
- Immutable one-writer releases

MY OWNERSHIP

Thomas set product goals, business definitions, safety invariants, release priorities, acceptance criteria, and owner-only decisions. He materially owned production diagnosis, implementation direction, regression strategy, release and rollback decisions, and evidence-driven handoff. Codex-assisted execution accelerated work under that direction.

VERIFIED ENGINEERING OUTCOMES

- | | | | |
|----|---|----|---|
| 01 | Deterministic stored values and explicit missing-versus-zero behavior | 02 | Same-item late-source refresh and immediate equivalent no-op |
| 03 | Accepted-write settlement by exact GET with no retry PUT | 04 | Serialized successor and rollback paths preserving one writer |